



● 第4章 软件架构的风格与模式

王璐璐 wanglulu@seu.edu.cn

廖力 lliao@seu.edu.cn

第4章 软件架构的风格与模式

- 4.0 引言
- 4.1 软件架构风格定义
- 4.2 软件架构风格的分类
- 4.3 典型的软件架构风格
- 4.4 软件架构模式

4.2 软件架构风格的分类

- (1) 数据流风格：批处理、管道/过滤器；
- (2) 调用/返回风格：主程序/子程序、面向对象风格、层次结构；
- (3) 以数据为中心的风格：仓库风格、超文本系统、黑板风格等；
- (4) 虚拟机风格：解释器、基于规则的系统；
- (5) 独立组件风格：进程通讯、事件系统。

虚拟机 (Virtual Machine)

你认为JAVA最吸引人的特性是什么？



Object Oriented

Automatic Memory Management

Security

Built-in Networking

Robust

Compiler/Interpreter Combo

Write Once, Run Anywhere

Dynamic Binding

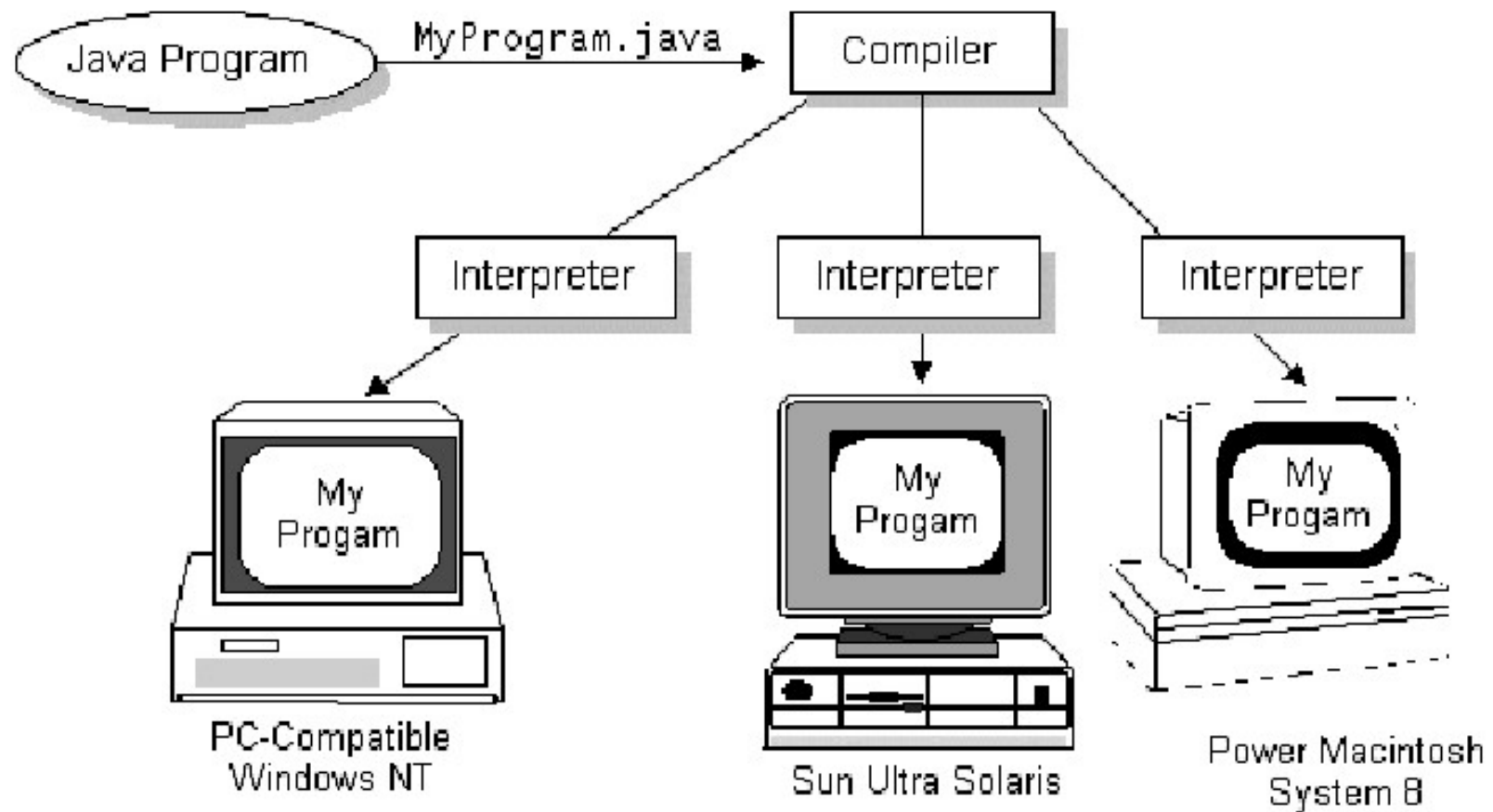
Platform Independence

Good Performance

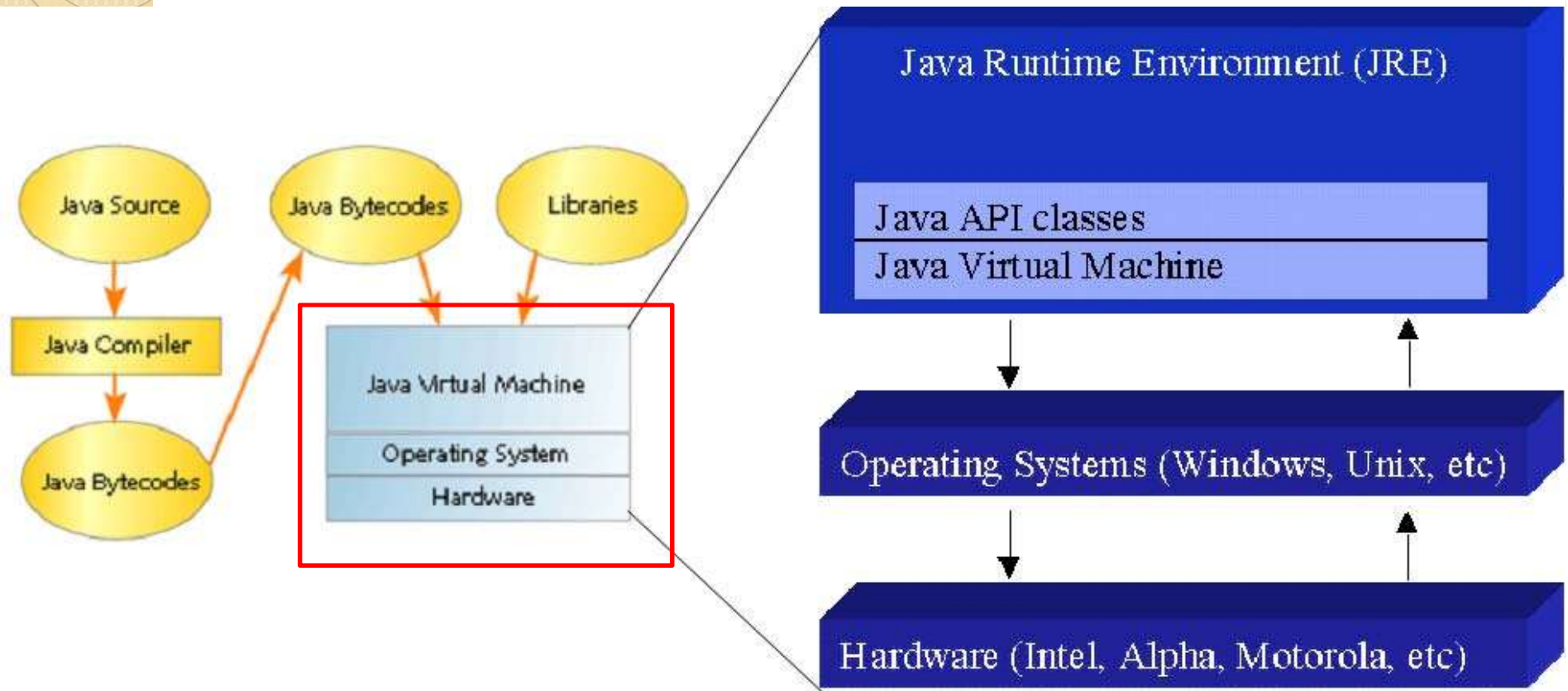
Several dangerous features of C&C++ eliminated

Write Once, Run Anywhere

一次书写，多处运行

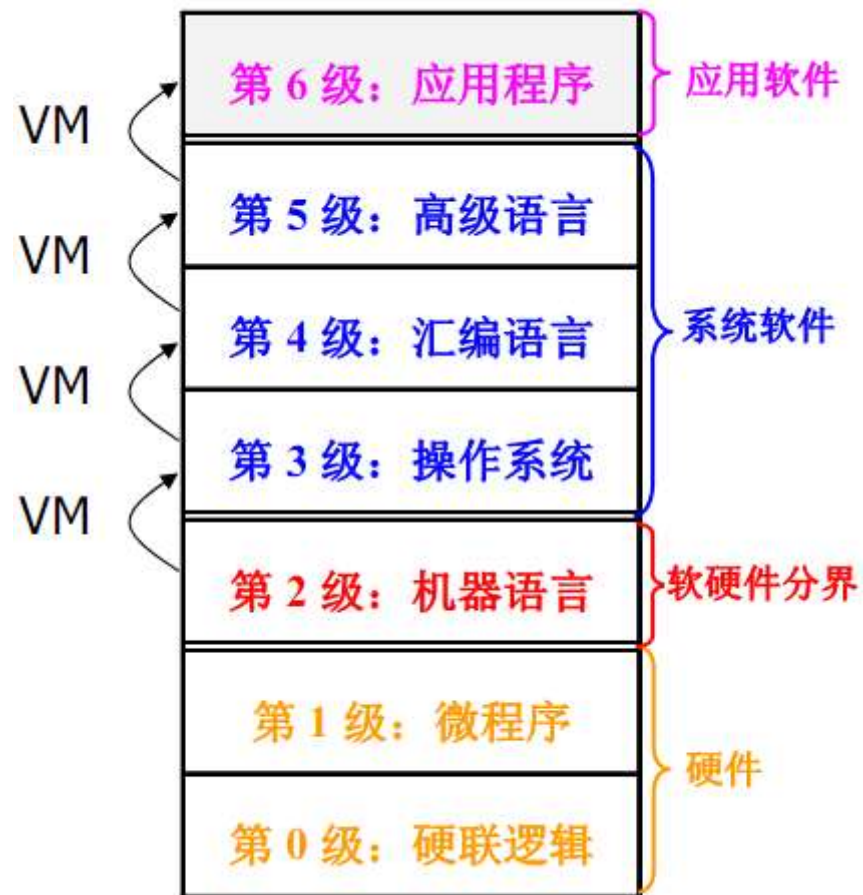


JAVA如何支持 Platform Independence



虚拟机 (Virtual Machine)

- A virtual machine is software that creates a virtualized environment between the computer platform and the end user in which the end user can operate software.
- 虚拟机是一种软件；
- 它创建了一种虚拟的环境；
- 将用户与底层平台隔离开来。

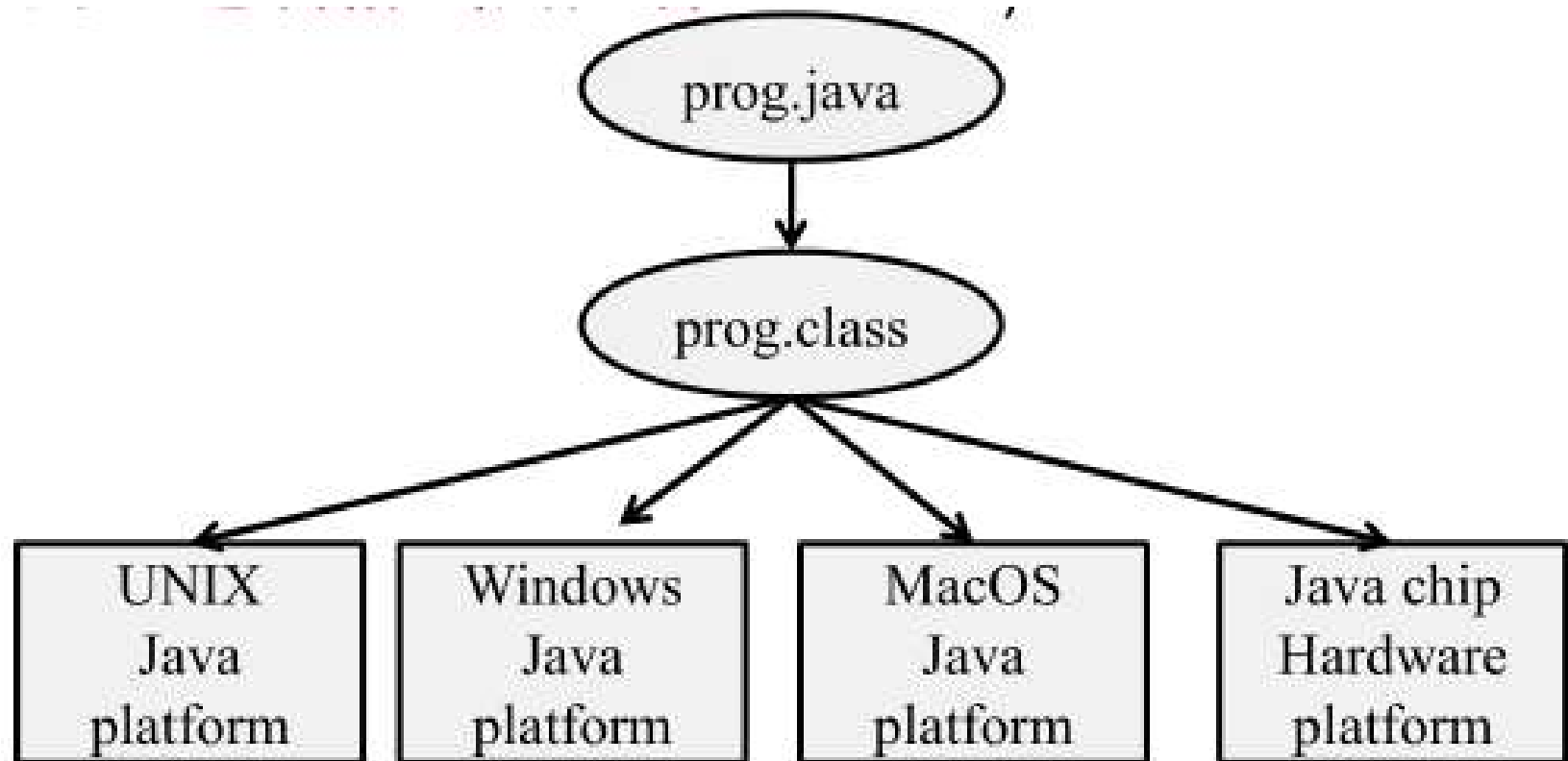


虚拟机 (Virtual Machine)

- 例如：A Java Virtual Machine (JVM), originally developed by Sun Microsystems, is a virtual machine that executes Java byteCode. (JVM: 执行Java字节码的虚拟机)
- JVM可适应所有的硬件与OS平台，从而使得Java具有“一次书写，到处运行”的能力

虚拟机 (Virtual Machine)

- 在JVM上运行的程序必须首先被编译为标准的二进制格式的文件：.class



虚拟机 (Virtual Machine)

- Java class文件并不是机器代码或目标代码，而是一种具有标准中间格式的二进制文件，无法直接在任何OS平台上执行；
- Java Class必须在JVM的支持下才能真正执行

虚拟机 (Virtual Machine)

- 不管何种类别的虚拟机，本质上都是在高层次抽象的用户与低层次抽象的OS/硬件之间建立一道屏障。
- 如何把上层应用的请求映射到下层OS/硬件系统的执行？
 - 解释器(Interpreter)
 - 基于规则的系统(Rule-based System)

4.3.6 解释器风格

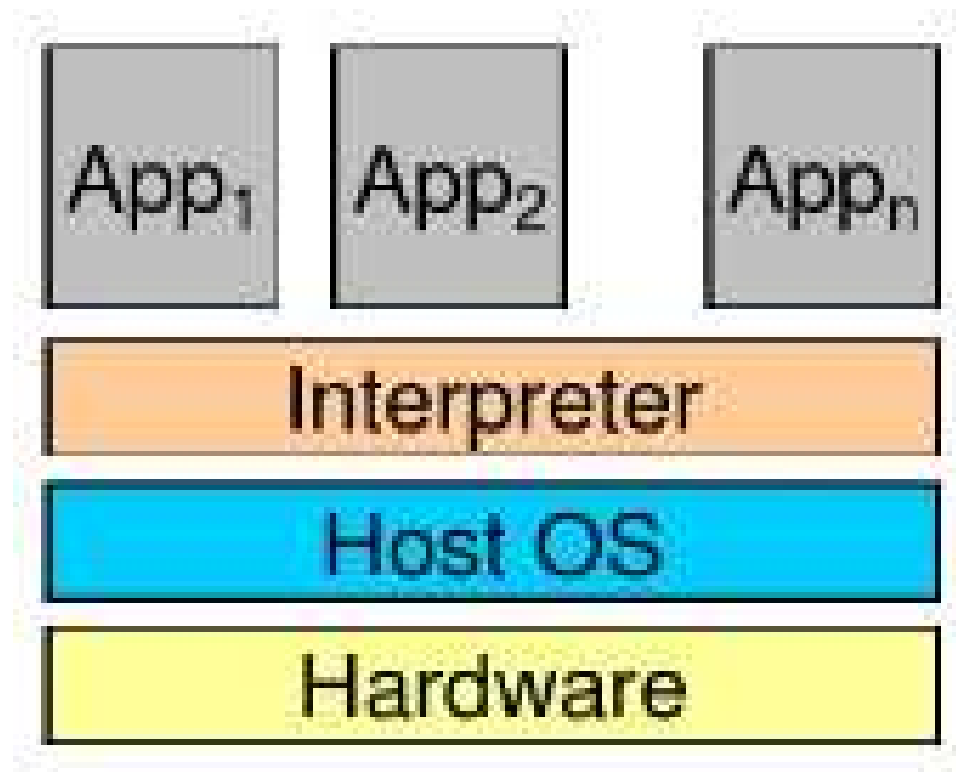
- 基本思想

- 解释器(Interpreter) 是一个用来执行其他程序的程序，它针对不同的硬件平台实现了一个虚拟机，**将高抽象层次的程序翻译为低抽象层次所能理解的指令**，以弥合程序语义所期望的与硬件提供的计算引擎之间的差距。



图4.14解释执行过程

4.3.6 解释器风格



4.3.6 解释器风格

- 组成部分

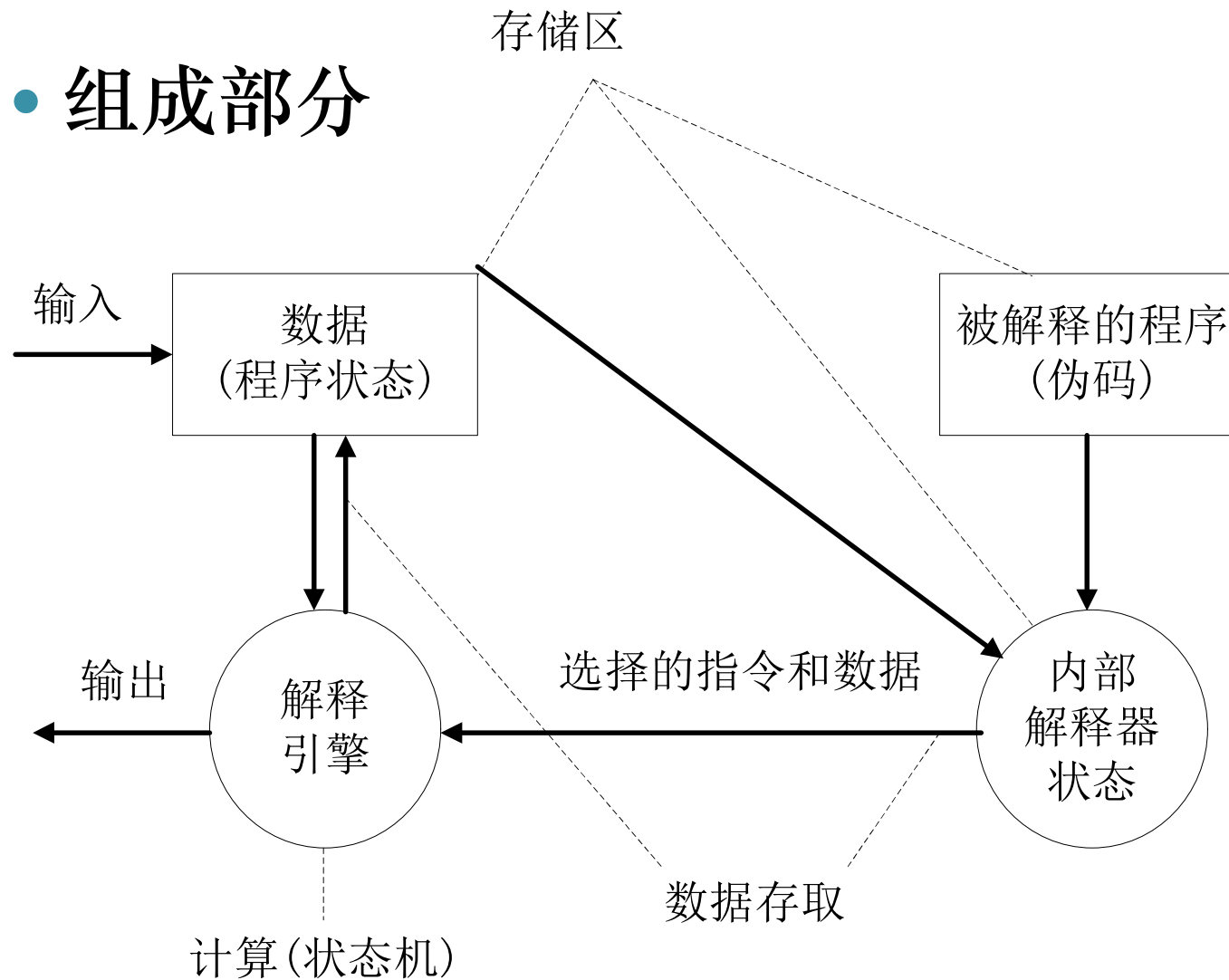


图4.15解释器风格

4.3.6 解释器风格

- 优点

- (1) 它有利于实现程序的可移植性和语言的跨平台能力；
- (2) 可以对未来的硬件进行模拟和仿真，能够降低测试所带来的复杂性和昂贵花费

4.3.6 解释器风格

- 缺点

- 额外的间接层次导致了系统性能的下降，如在不引入JIT（Just In Time）技术的情况下，Java应用程序的运行速度相当慢。

4.3.6 解释器风格

- 应用实例

- JVM

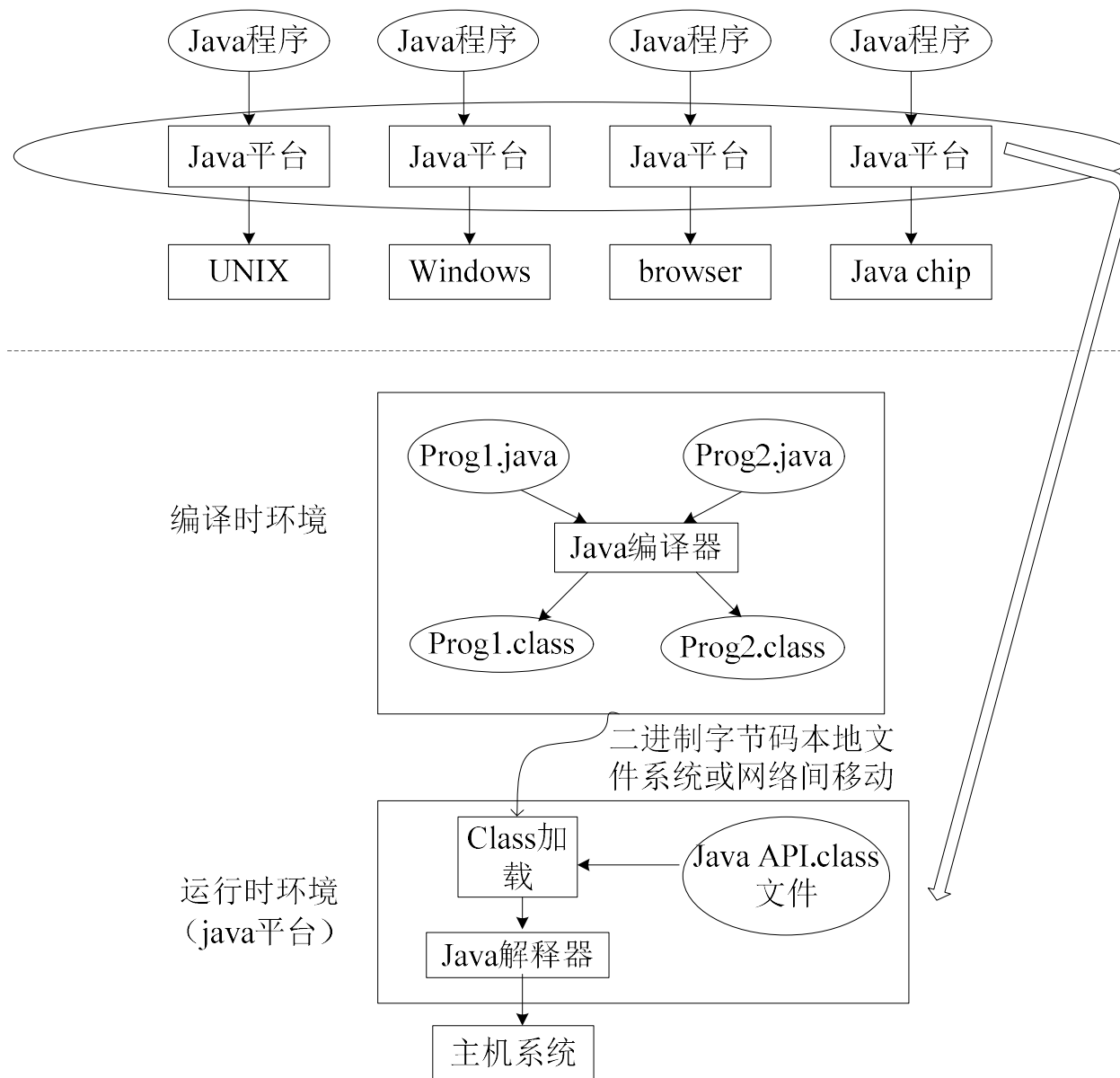


图4.16基于JVM的
Java程序执行过程

解释器 vs. 编译器

- 目标
- 功能
- 性能
- 实现

解释器 vs. 编译器

- **编译器**不会执行输入的源程序代码，而是将其翻译为另一种语言，通常是可执行的机器码或目标码，并输出到文件中以便随后链接为可执行文件并加以执行

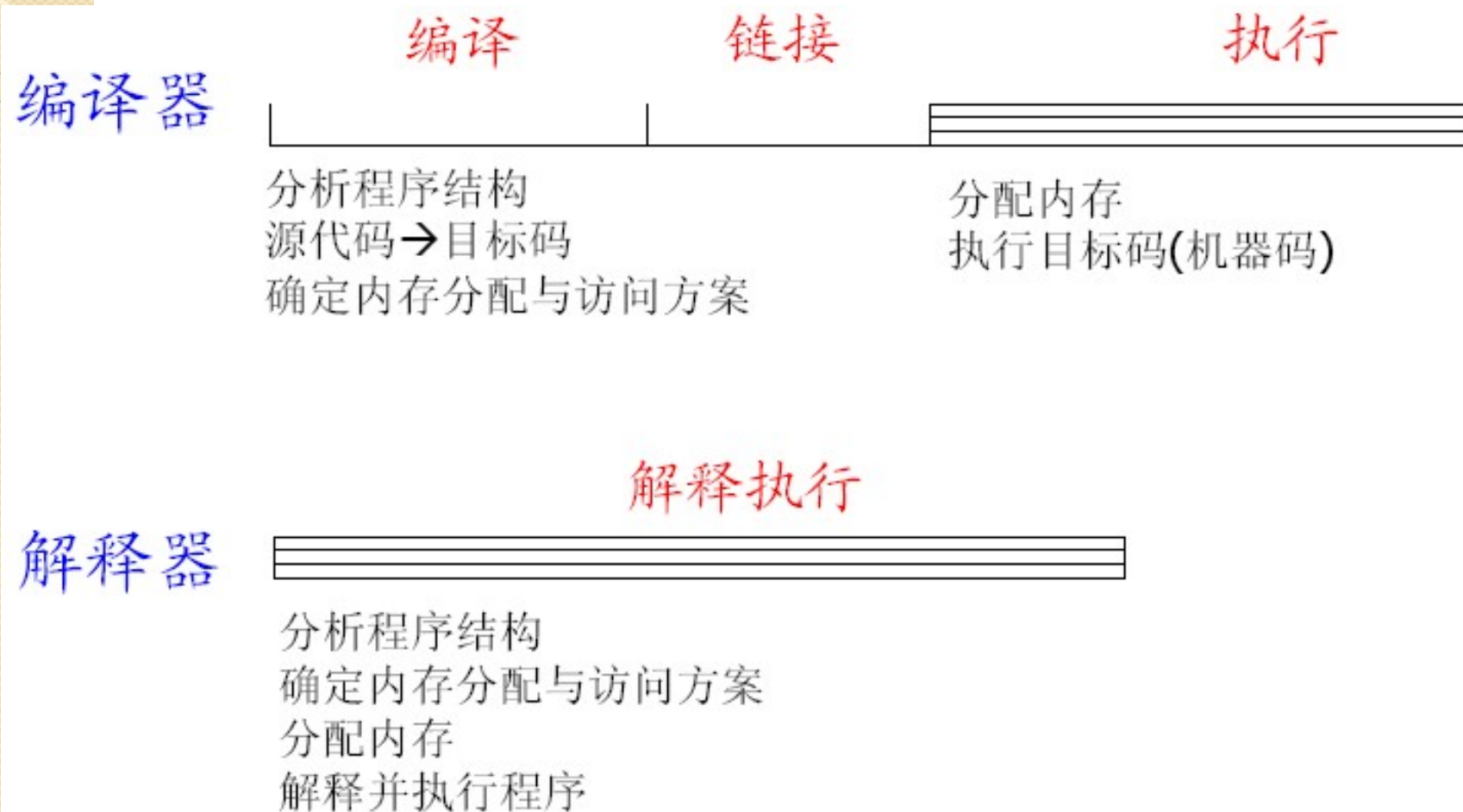


解释器 vs. 编译器

- 在解释器中，程序源代码被解释器直接加以执行。



解释器 vs. 编译器



解释器 vs. 编译器

- 解释器的执行速度要慢于编译器产生的目标代码的执行速度，但是却低于编译器“编译+链接+执行”的总时间
- 解析器执行速度之所以慢，是因为每次解释执行的时候，都需要分析程序的结构，而编译代码则直接执行而无需重复编译
- 解释器对内存的分配是在解释时才进行的；而编译器则是在编译时就规划好了变量的内存使用方案，因此运行时直接将程序代码装入内存并执行即可

解释器的三种策略

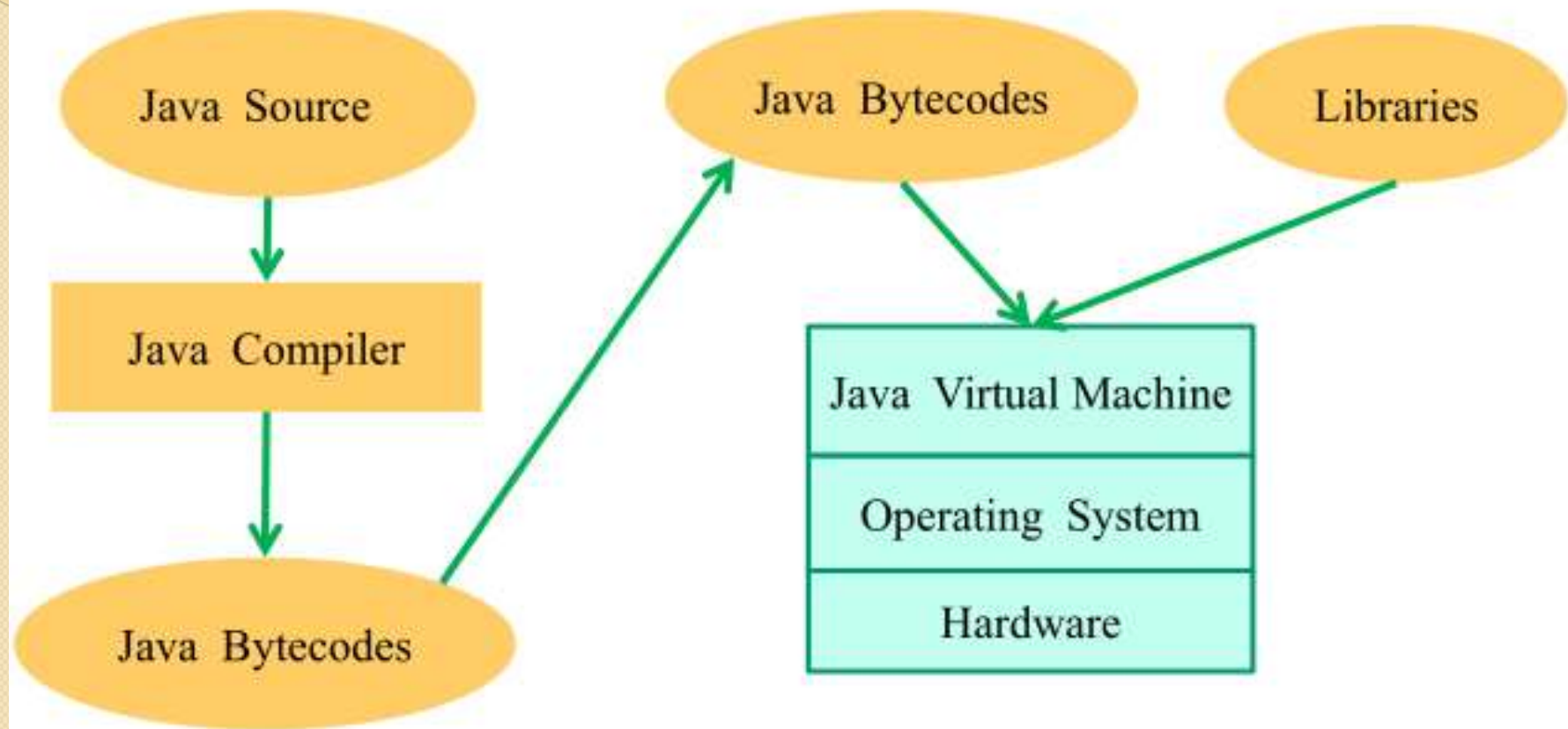
- 传统解释器(traditionally interpreted)
 - 纯粹的解释执行
- 基于字节码的解释器(compiled to bytecode which is then interpreted)
 - 编译-> 解释执行
- Just-in-Time (JIT)编译器
 - 编译||解释执行

传统解释器

- 解释器直接读取源代码并加以执行；
 - ASP
 - Excel
 - JavaScript
 - MATLAB
 - etc

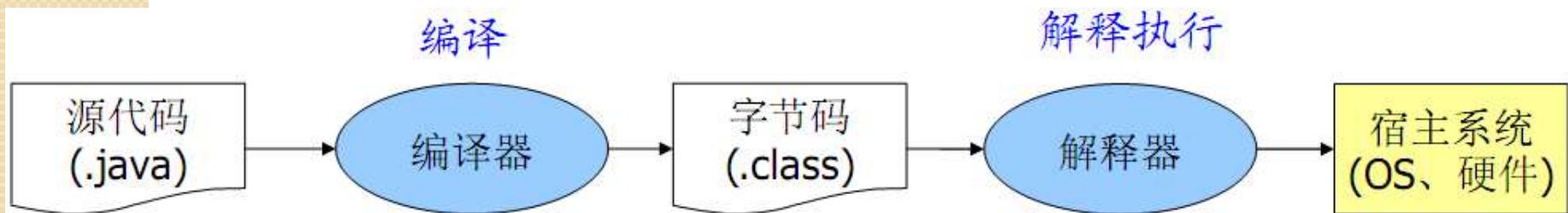


字节码解释器 (Bytecode interpreter)



字节码解释器 (Bytecode interpreter)

- 源代码首先被“编译”为高度压缩和优化的字节码，但并不是真正的机器目标代码，因而与硬件平台无关；
- 编译后得到的字节码然后被解释器加以解释；
 - 例如：Java, Perl, PHP, Python

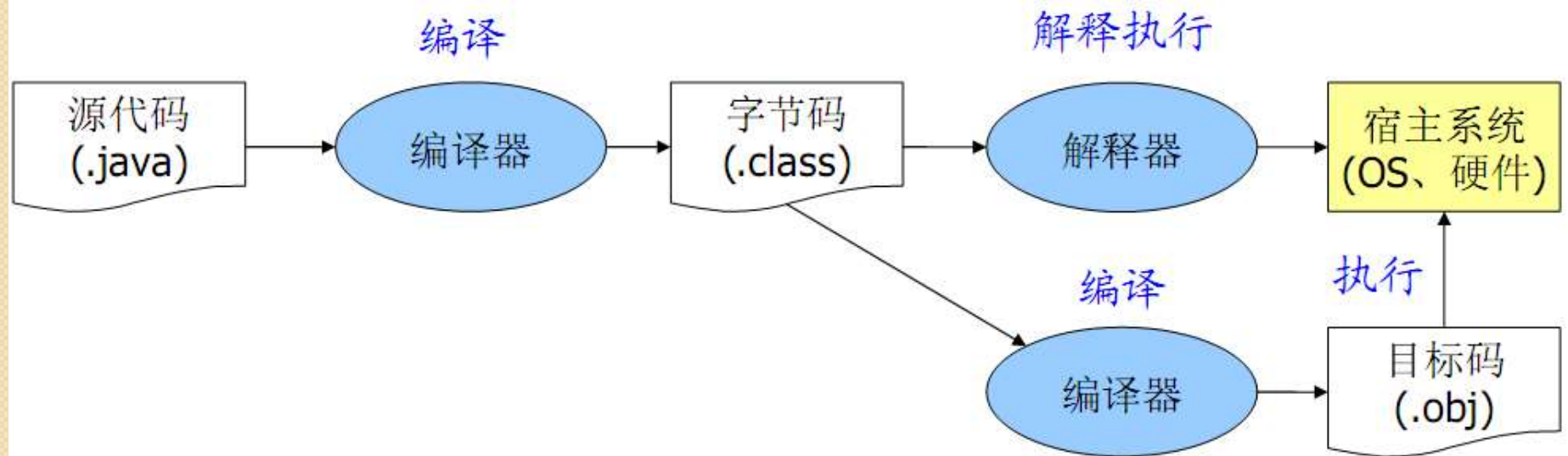


实时编译器JIT (Just-in-time) Compiler

- 实时编译JIT中，字节码在运行时被编译为本机的目标代码
 - 实时编译JIT中，第一步编译得到字节码；
 - 然后，字节码被配置到目标系统中；
 - 当字节码被执行时，运行环境下的编译器将其翻译为本地机器码

实时编译器JIT (Just-in-time) Compiler

- JIT模糊了解释器、字节码解释器和编译器之间的边界与区分
 - JIT is available for both the .NET and Java platforms.



思考题

- 针对解释器的三种策略：
 - 传统解释器(traditionally interpreted)
 - 基于字节码的解释器 (compiled to bytecode which is then interpreted)
 - Just-in-Time (JIT)编译器
- 对比各自的工作原理，并对比分析各自的优缺点。

4.3.7 基于规则的系统风格

IF Customer.age<18
AND Withdraw.amount>1000
THEN *signature of the parent is required*

如果客户年龄小于18岁，
而且取款金额大于1000元，
那么，需要家长的签名；

IF CustomerOrder.TotalMoney >100
THEN CustomerOrder.Discount = 10%

如果客户订单的总金额大于100元，
那么，该订单的折扣为10%；

IF Order.Sum > ¥ 10,000
AND Order.Sum <= ¥ 50,000
THEN Order.AuditedBy("DepartmentManager")

如果订单总金额大于1万人民币，
而且订单总金额小于5万人民币，
那么，该订单需要由部门经理审核。

4.3.7 基于规则的系统风格

- 问题

- 现实中的业务需求通常采用自然语言来表达，自然语言和程序代码之间存在“语义鸿沟”
- 有些业务逻辑无法表达为if-else结构
- 现实里的业务需求经常频繁的发生变化，每次需求变化都要修改代码的话，效率低，成本高

4.3.7 基于规则的系统风格

- 现实里的业务需求经常频繁的发生变化，不断修改代码效率低、成本高。**最好的办法是把频繁变化的业务逻辑抽取出来，形成独立的规则库。**
- 这些规则可独立于软件系统而存在，可被随时的更新。
- 系统运行时，读取规则库，依据当前运行状态，从规则库中选择与之匹配的规则，对规则进行解释，根据结果控制系统运行的流程。

4.3.7 基于规则的系统风格

- 业务逻辑=固定业务逻辑+可变业务逻辑(规则)+规则引擎
- 基于规则系统 (Rule-based System)
 - 一个使用模式匹配搜索来寻找规则并在正确的时候应用正确的逻辑知识的虚拟机。
 - 基于规则系统提供了一种基于专家系统解决问题的手段，将知识表示为“条件-行为”的规则，当满足条件时触发相应的行为，而不是将这些规则直接写在程序源代码中。

4.3.7 基于规则的系统风格

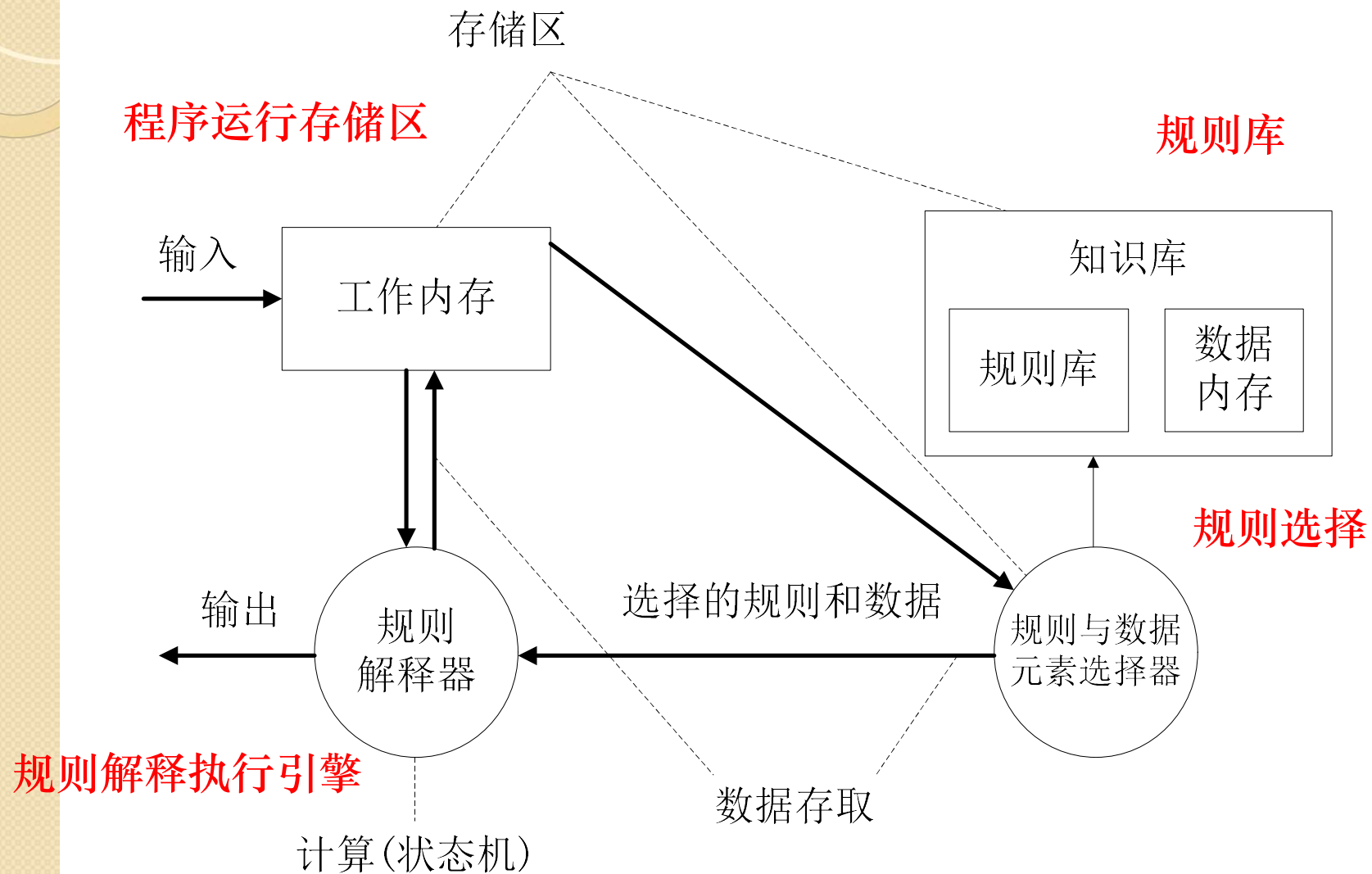


图4-19基于规则的系统

4.3.7 基于规则的系统风格

- 核心思想
 - 将业务逻辑中可能频繁发生变化的代码从源代码中分离出来
- 基本过程
 - 使用规则定义语言（如：基于XML）将这些变化部分定义为“规则”；
 - 在程序运行时，规则引擎根据程序执行的当前状态，判断需要调用并执行哪些规则，进而通过“解释器”的方式来解释执行这些规则。
 - 其它直接写在源代码中的程序，仍然通过传统的“编译”或“解释”的办法加以执行。

4.3.7 基于规则的系统风格

- 规则的表达形式
 - 任何规则都包含两部分：
 - IF 部分：规则的前提或条件
 - THEN部分：规则的结论或触发的行为
 - 一个规则可以有多个条件，使用AND或OR连接

4.3.7 基于规则的系统风格

- 规则示例

IF Customer.age<18
AND Withdraw.amount>1000
THEN *signature of the parent is required*

如果客户年龄小于18岁，
而且取款金额大于1000元，
那么，需要家长的签名；

IF CustomerOrder.TotalMoney >100
THEN CustomerOrder.Discount = 10%

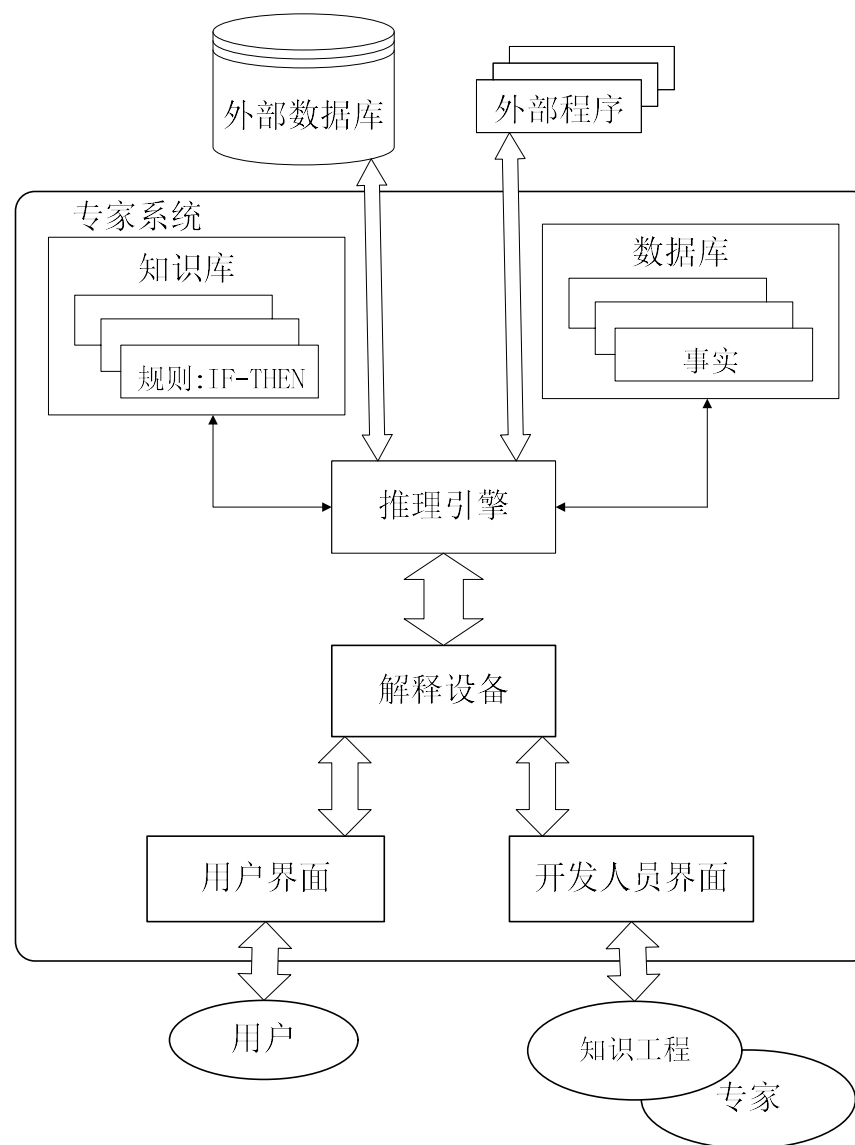
如果客户订单的总金额大于100元，
那么，该订单的折扣为10%；

IF Order.Sum > ¥ 10,000
AND Order.Sum <= ¥ 50,000
THEN Order.AuditedBy("DepartmentManager")

如果订单总金额大于1万人民币，
而且订单总金额小于5万人民币，
那么，该订单需要由部门经理审核。

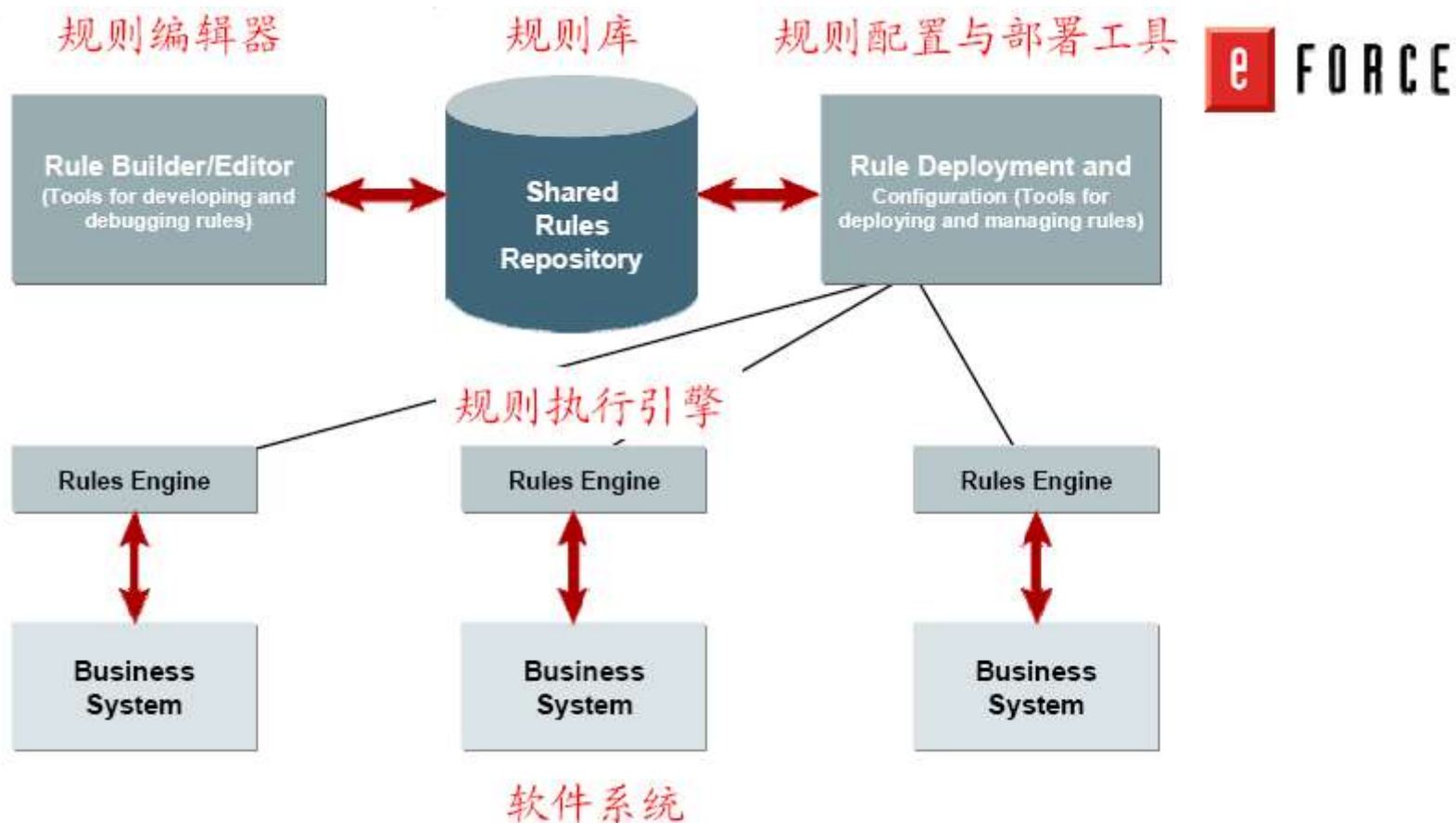
4.3.7 基于规则的系统风格

- 应用实例
 - 基于规则的专家系统



4.3.7 基于规则的系统风格

- 应用：业务规则引擎



4.3.7 基于规则的系统风格

- 优点：
 - 降低了修改业务逻辑的成本
 - 缩短了开发时间
 - 将规则外部化，可在多个应用之间共享
 - 对规则的改变将非常迅速并且具有较低的风险

4.3.7 基于规则的系统风格

- 基于规则的系统风格的基本组件与解释器风格的组件相似

基于规则的系统	解释器风格
知识库（规则库）	待解释的程序（伪码）
规则解释器	解释器引擎
规则与数据元素选择	解释器引擎内部的控制状态
工作内存	程序当前的运行状态

4.3.7 基于规则的系统风格

- 基于规则的系统和解释器风格的系统比较

	基于规则的系统	解释器风格的系统
相同点	基于规则的系统本质上是与解释器风格一致的，都是通过“解释器”(“规则引擎”)，在两个不同的抽象层次之间建立起一种虚拟的环境。	
不同点	在自然语言/XML的规则和高级语言的程序源代码之间建立虚拟机环境	在高级语言程序源代码和OS/硬件平台之间建立虚拟机环境



THANKS